Kiss-50 Differences from Kiss-100

Status: Working public/reference draft.
Scope: This document explains the structure of Kiss-50 and the practical differences from Kiss-100. It intentionally does **not** duplicate the full ISA and assembler manuals.
Last verified: 2026-06-18 against simulator/Kiss10.db tables KS50Reg, KS50ISA, KS50Asm, and KS50AsmOpr.

1. Purpose

Kiss-50 is the compact 50-bit member of the Kiss architecture family. It targets smaller, embedded, or more resource-constrained systems while preserving the core Kiss model:

- 10-bit bytes
- one-based numbering
- typed arithmetic
- fixed-width instructions
- load/store operation
- conditional execution blocks
- the same broad assembler language concepts used by Kiss-100

Kiss-50 is **not a different programming language**. It is a compact architectural target with a reduced instruction encoding and reduced hardware footprint.

For the full language and command set, see the existing shared references:

- docs/isa/kiss-isa-reference.md
- docs/isa/kiss-isa-manual-source.md
- docs/isa/kiss-isa-commands-source.md
- docs/assembler-language/kiss-assembler-grammar.md
- docs/assembler-language/kiss-assembler-manual-source.md
- docs/assembler-language/kiss-assembler-commands-source.md

This document focuses only on the differences a reader needs to understand when targeting Kiss-50 instead of Kiss-100.

2. High-Level Comparison

Feature	Kiss-100	Kiss-50
Instruction width	100 bits	50 bits
Instruction storage	10 Kiss bytes	5 Kiss bytes
Address width	50 bits	30 bits
Opcode field	12 bits	10 bits
CMB field	8 bits	5 bits
Operand slots	5 × 15-bit operands	3 × 10-bit operands
Register ID width	15 bits	10 bits
Register ID range	1–32767	1–1023
General-purpose banks	R1–R5	R1 mandatory, R2 optional
Data stacks	10	1
Executable format	`.kexe`	`.k5exe`
Object format	`.kobj`	`.k5obj`
Main simulator/UI target	K10VM	K5VM

The most important idea is simple:

Kiss-50 keeps the Kiss programming model, but compresses the encoding and reduces the architectural footprint.

That means many familiar commands and concepts still exist, but some have fewer operands, some require explicit lowering, and some Kiss-100 forms do not fit in the compact format.

3. Instruction Format

For direct comparison, the two base layouts are:

```
Kiss-100:
CEB 5 | Command 12 | CMB 8 | Opr1 15 | Opr2 15 | Opr3 15 | Opr4 15 | Opr5 15

Kiss-50:
CEB 5 | Opcode 10 | CMB 5 | Opr1 10 | Opr2 10 | Opr3 10
```

Kiss-50 instructions are exactly 50 bits:

```
50 bits total:
CEB 5 | Opcode 10 | CMB 5 | Opr1 10 | Opr2 10 | Opr3 10
```

Field	Bits	Meaning
CEB	5	Conditional Execution Block field. Conceptually the same role as Kiss-100.
Opcode	10	Compact Kiss-50 command number. Opcode `0` is invalid.
CMB	5	Compact command modifier bits. Reduced from Kiss-100’s 8-bit CMB.
Opr1	10	Operand slot 1.
Opr2	10	Operand slot 2.
Opr3	10	Operand slot 3.

Consequence of the compact format

Kiss-100 has five 15-bit operand slots. Kiss-50 has only three 10-bit operand slots.

Because of that:

- some Kiss-100 forms survive directly,
- some are compacted to a smaller operand model,
- some require explicit preparatory instructions,
- and some surface forms are intentionally not active in Kiss-50.

4. Register Structure

4.1 Register ID space

Kiss-50 uses 10-bit register IDs.

- Valid ID range: 1–1023
- 0 is invalid/reserved as an error sentinel

This is much smaller than Kiss-100, so Kiss-50 cannot simply carry forward every Kiss-100 register view and alias.

4.2 Current verified active register counts

Verified from KS50Reg on 2026-06-18:

- **924 active registers**
- active ID range: **1–992**

4.3 Current register layout

Range	Count	Assignment
`1–437`	437	R1 bank
`438–460`	23	R1 expansion reserve
`461–887`	427	R2 bank
`888–920`	33	R2 expansion reserve
`921–948`	28	System registers
`949–960`	12	System expansion reserve
`961–965`	5	R8 scratch pool
`966–970`	5	Stack 1 registers
`971–992`	22	R8UA overlays / subregisters

Range	Count	Assignment
`993–1023`	31	Reserved

4.4 General-purpose bank differences

Kiss-100 uses a larger bank model (R1–R5). Kiss-50 reduces that sharply:

- **R1 is mandatory**
- **R2 is optional**
- higher full general-purpose banks are not part of the compact target

This makes Kiss-50 better suited for smaller implementations while keeping the familiar register naming style where possible.

4.5 Width policy differences

Kiss-50 does **not** preserve every Kiss-100 width/view.

Surviving R1 widths

R1 keeps:

- 2000
- 1000
- 500
- 400
- 300
- 200
- canonical 100-bit views
- 50
- 40
- 30
- X/Y 20-bit views
- 10-bit views

Surviving R2 widths

R2 keeps:

- 2000
- 1000
- 500
- 400
- 300
- 200
- canonical 100-bit views
- 90
- 80
- 70
- 60
- 50
- 30
- 10-bit views

Important difference:

- **R2 has no 20-bit register family in Kiss-50.**

Removed / collapsed width families

Kiss-50 removes a number of Kiss-100 views in order to fit within the compact register-ID budget.

Examples of removed or not-carried-forward patterns include:

- 5-bit GP register families
- several intermediate or less essential width variants
- many alias-style duplicate views

4.6 Canonical 100-bit naming only

Kiss-50 keeps the canonical 100-bit naming style:

- R?WA through R?ZE

Alias-heavy alternatives from Kiss-100 are removed.

4.7 Decimal-register difference

Kiss-50 does **not** expose the Kiss-100 style 5-bit decimal digit general-purpose register families.

Decimal arithmetic still exists as a data-type concept, but Kiss-50 does not preserve the large GP decimal-digit register inventory of the full architecture.

4.8 Address-bearing registers are 30-bit in Kiss-50

Important address-bearing/system registers are compacted to 30-bit address width in KS50Reg, including verified examples such as:

- PC
- BASE
- BOUNDS
- VM_TABLE_BASE
- MSG_SEND
- MSG_RECV
- EXCEPTION_PC
- SP1
- SB1
- SL1
- SFP1
- SSP

This matches the Kiss-50 30-bit addressing model.

4.9 R8 scratch pool and R8UA overlays

Kiss-50 keeps a compact scratch strategy:

- R8UA through R8UE as a small scratch pool
- additional **R8UA overlays/subregisters** in the 971–992 range for compact literal materialization and related lowering work

This is one of the key practical differences from Kiss-100: Kiss-50 relies more heavily on compact staging/lowering through scratch and overlay registers when a direct instruction form does not fit.

5. Operand Model Differences

Kiss-50 uses the separate KS50AsmOpr table for compact operand-family validation.

Current verified counts:

- KS50AsmOpr: 40 total rows
- 36 active rows

The compact operand model typically uses patterns like:

Shape	Meaning
`src src dest`	two inputs, one destination
`value target`	in-place update target
`target`	single target or single destination
`count addr dest`	compact memory form
`target - -`	control-transfer style

Compared with Kiss-100, Kiss-50 generally avoids:

- wide fanout destination forms
- 4-operand offset memory forms
- 5-operand instruction forms

When a Kiss-100 operation would need more than three architectural operand slots, Kiss-50 generally expects software or assembler lowering to break the work into smaller explicit steps.

6. CMB Differences

6.1 Reduced from 8 bits to 5 bits

Kiss-100 uses an 8-bit CMB field. Kiss-50 reduces this to 5 bits.

That does **not** mean “all modifier behavior is gone.” It means the modifier layout is compacted and often family-specific.

6.2 Default compact CMB pattern

For many families, the compact CMB roughly preserves:

- data type selection
- set-flags behavior
- one family-specific behavior bit
- one additional family-specific behavior bit

Typical compact uses include:

- data type
- set status flags
- saturation
- left/right or direction
- relative/backward control-flow mode
- immediate-vs-register selector for a few compact cases such as Incr/Decr

6.3 Family-specific remapping matters more in Kiss-50

Because the field is smaller, CMB meaning is more tightly packed and more dependent on instruction family.

An important consequence is that some Kiss-100 behaviors that had dedicated modifier room become more constrained in Kiss-50. In particular, Kiss-100 jump/call encodings had more room for control modifiers such as branch/conditional-exit handling. Kiss-50 does not have spare space to carry every Kiss-100 control-transfer modifier bit forward unchanged.

Public-facing takeaway: readers should not assume Kiss-100 control-transfer CMB details map 1:1 into Kiss-50. Where a wide Kiss-100 control-flow nuance does not fit directly, the compact target may represent it differently, lower it through other forms, or document it only in the detailed assembler/implementation references.

Readers should rely on:

- the assembler command reference,
- the active KS50Asm rows,
- and the Kiss-50 examples in K5VM samples,

rather than assuming every Kiss-100 CMB bit carries over unchanged.

6.4 Compact SetImm CMB

Current verified assembler rows show:

- SetVal base CMB = 00100
- SetTxt base CMB = 00110

This compact format preserves typed-literal meaning while fitting inside the 5-bit Kiss-50 modifier field.

7. Command / Form Differences

7.1 Most concepts survive, but not every Kiss-100 surface form

Kiss-50 currently verifies as:

- KS50ISA: **179 active ISA rows**
- KS50Asm: **590 total rows, 582 active rows**

That means Kiss-50 is broad and real, not a toy subset. But it is still selective.

7.2 Commands that compact cleanly

Many common operations survive well in three-slot form, including:

- arithmetic
- logic
- shifts/rotates
- bit operations
- base memory load/store
- register moves/copy/clear/swap
- compare/test
- jump/call/if/end-if/return families
- I/O
- key system operations

7.3 Compact memory model

Base memory forms survive, but wide offset forms do not.

In particular, Kiss-100-style surface forms such as:

- LoadOfs
- LoadOfsMul
- StorOfs
- StorOfsMul

are not active Kiss-50 assembler forms.

Instead, Kiss-50 expects software to:

1. compute the effective address explicitly, then
2. use the base Load / Stor form.

7.4 Compact MulAdd still exists

MulAdd remains valid in Kiss-50 even though there are only three operand slots.

Its compact meaning is:

```
Opr3 = Opr1 × Opr2 + old(Opr3)
```

So operand 3 is both:

- the incoming accumulator/addend source, and
- the destination register

This is a deliberate compact encoding choice, not an omission.

7.5 Control-transfer differences

Kiss-50 keeps jump/call/if structures, but direct immediate/label encodings are smaller because operand fields are only 10 bits wide.

Practical consequence:

- very small direct targets may encode directly,
- larger control-flow targets should be materialized into an address-capable register and then jumped/called through that register.

8. Compact SetImm and Literal Strategy

Kiss-100 has a roomier immediate story. Kiss-50 uses a compact two-part payload.

8.1 SetImm format

Kiss-50 SetImm uses:

```
Opr1 = destination register
Opr2 = payload high 10 bits
Opr3 = payload low 10 bits
```

This provides a **20-bit compact payload**.

8.2 Why this matters

A 20-bit payload is enough for many common small literals, but not for all values or all address

materialization cases.

So larger values are generally handled by:

- staging through compact SetImm chunks,
- using R8UA overlay/subregister mechanisms,
- then referencing the assembled wider value through a target register.

That compact literal-lowering strategy is one of the defining practical differences between Kiss-50 and Kiss-100.

9. Stack Differences

Kiss-100 has 10 stacks. Kiss-50 has only **1** stack.

Verified stack-related Kiss-50 register set:

- SP1
- SB1
- SL1
- SFP1
- SSP

This means code that assumes the broader Kiss-100 multi-stack environment must be adapted for the compact target.

10. Executable / Object Format Differences

Kiss-50 uses compact target-specific binary containers:

- executable format: .k5exe
- object format: .k5obj

The compact executable uses its own record/packing assumptions and should not be treated as a simple .kexe rename.

MODULE_CONFIG remains part of the architecture/module-discovery model and is shared as a required-modules concept in the compact binary/header flow as well.

11. Module / Profile Differences

Kiss-50 is still part of the broader Kiss modular architecture.

It keeps the same general module philosophy:

- Base
- Decimal
- System
- Atomic
- AI/Tensor
- future/optional modules and profiles as defined by the architecture

But Kiss-50 should be thought of as the **compact base architecture**, not just Kiss-100 with fewer registers.

A few practical profile ideas already reflected in the design include:

- R1-only vs R1+R2 implementations
- smaller embedded-style hardware footprints
- compact loaders and compact executable/object formats

12. What a Reader Should Take Away

If you already know Kiss-100, the safest mental model is:

1. **The language is mostly familiar.**
2. **The hardware envelope is smaller.**
3. **The encoding is much tighter.**
4. **Some wide Kiss-100 forms must be lowered explicitly.**
5. **Register views are more selective and compact.**
6. **Literal/address materialization matters more.**

Kiss-50 is therefore best seen as a compact, deployment-oriented member of the Kiss family rather than a separate ecosystem.

13. Related References

For full details, use this document together with:

- projects/Kiss/kiss50/kiss50-spec.md
- projects/Kiss/kiss50/kiss50-registers.md

- projects/Kiss/kiss50/kiss-modules.md
- projects/Kiss/kiss50/K5VM/README.md
- projects/Kiss/kiss50/K5VM/samples/
- projects/Kiss/docs/isa/kiss-isa-reference.md
- projects/Kiss/docs/assembler-language/kiss-assembler-grammar.md
- projects/Kiss/docs/assembler-language/kiss-assembler-commands-source.md

14. Suggested Website Summary Blurb

If a short public-facing summary is needed for the website, this wording is a reasonable starting point:

Kiss-50 is the compact 50-bit Kiss ISA target for embedded and constrained systems. It preserves the core Kiss programming model and assembler style while reducing instruction width, register-ID space, operand count, stack count, and address width compared with Kiss-100. See the Kiss-50 differences/reference document for the compact register layout, command-form differences, and literal/materialization rules specific to the K5 target.